

⇒ Word Embeddings

* Representation of word for word analysis typically in the form of real valued vector that encodes the meaning of word, such that words that are close in vector space are expected to be similar in meaning.

⇒ I have two words.

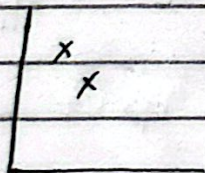
(1) Happy

(2) Excited

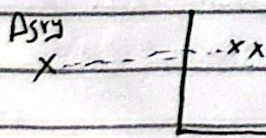
vectors → $\begin{bmatrix} \\ \end{bmatrix}$

$\begin{bmatrix} \\ \end{bmatrix}$

In 2D space these two words will be near



Now we have "angry" opposite of happy



• very far from happy

⇒ Word Embedding Techniques

Count or Frequency

- (1) OHE
- (2) BOW
- (3) TF-IDF

Deep Learning Trained Model

• Word 2 Vec
CBOW Skipgram

⇒ Word 2 Vec:

- Deep Learning Trained Model
- published by google in 2013.

* Use neural Network Model to learn word association from large corpus of text.

→ Once Trained, such a model can detect synonymous words or suggest additional words for a particular sequence

→ Suppose I have Vocabulary (Unique words).

* Each & every word will be converted into feature representation.

⇒ Boy Girl King Queen Apple Mango

→ This whole vector represent "Boy".

Gender	-1	+1	-0.92
Royal	0.001	0.02	0.95
Age	0.03	0.02	0.75
Food	-	-	-
...	-	-	-
him	-	-	-

→ Vector for King

* (King is royal so high relation, also king may be old high relation with age)

→ We have no sentence saying "he is a royal boy" so with royal very less relation of "boy" with "royal"

features

(i.e 300 features)

*** Each word will be represented by 300 features.

*** Every word will be having 300 dimension of vector

* In Cosine: $\rightarrow$ Distance Near "0" similar Vectors.
 $\rightarrow$ Distance Near "1" Different Vectors.

* We are creating Feature Representation of every word.

* Based on Relation of word with feature we get value.

$\rightarrow$ Now Vector addition or subtraction

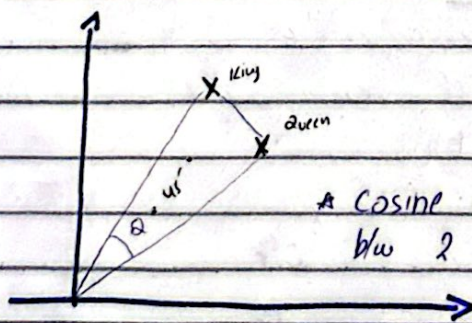
* King $\rightarrow$ boy + Queen = Girl

$\rightarrow$ Suppose These vectors

$$\begin{aligned} \text{King} &= [0.95, 0.96] & \text{MAN} &= [0.95, 0.98] \\ \text{Queen} &= [-0.96, 0.95] & \text{woman} &= [-0.94, -0.96] \end{aligned}$$

$$\text{King} - \text{MAN} + \text{Queen} = \text{Woman}$$

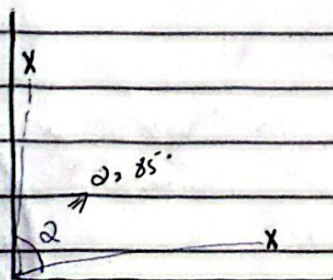
$\rightarrow$ Cosine Similarity:



$$\text{Distance} = 1 - \text{cosine similarity}$$

$$\text{Distance} = 1 - \cos 45^\circ = 0.29$$

* cosine similarity is distance Angle b/w 2 vectors



$$\begin{aligned} \text{so Distance} &= 1 - \cos 85^\circ \\ &= 1 - 0 \\ &= 1 \\ &\downarrow \\ &\text{No similarity} \end{aligned}$$

vector $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ → This 0 represent size
so vector of size 2

CBOW (Fully Connected Layered N/w)

* Input

Layer

(Take 4

Words, of vector
size 7)

* Hidden

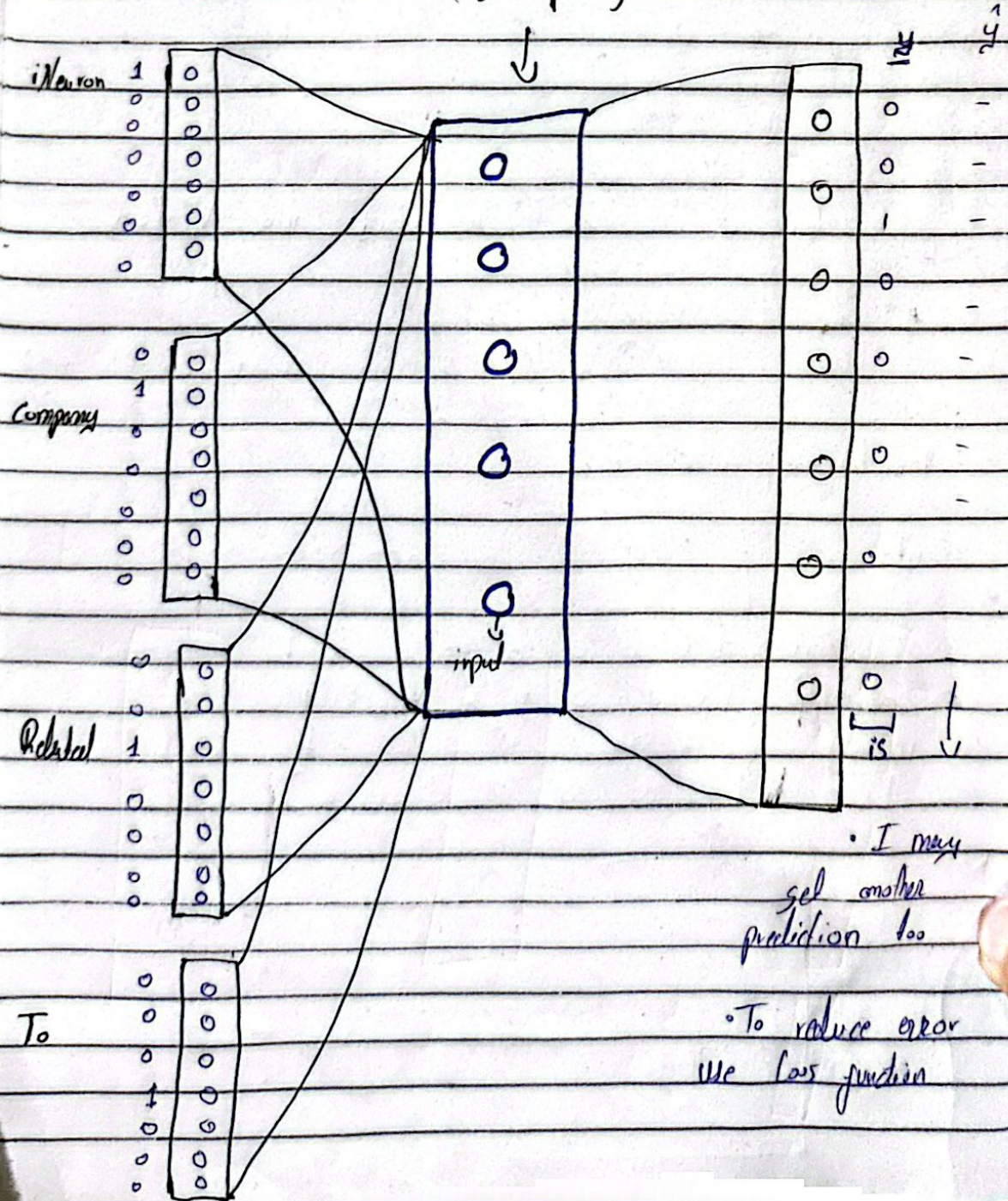
Layer

(Window size
& will have
5 input)

* Output

Layer

(of size
vector = 7)



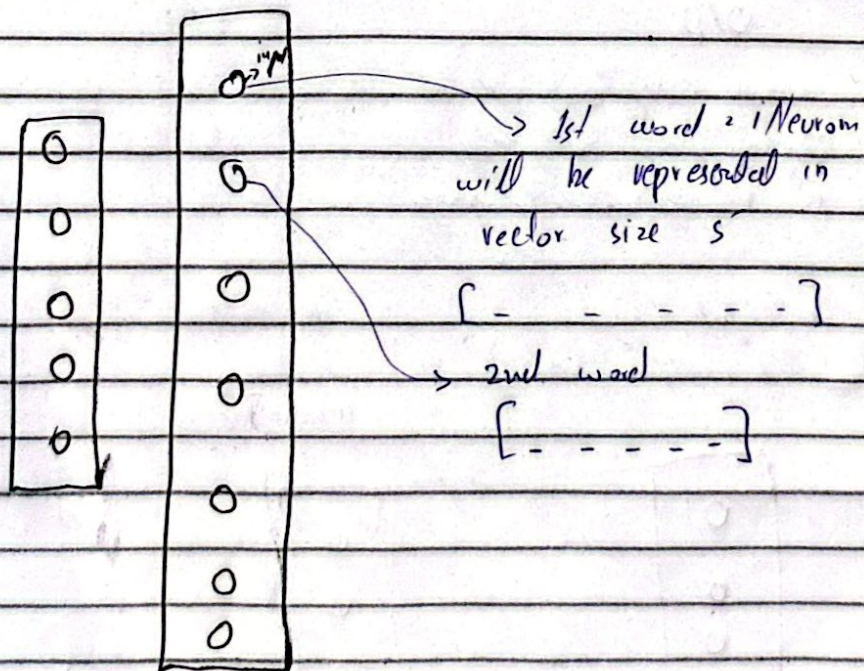
* Window size

* Feature representation of
vector size.

→ window size = 5 ⇒ feature representation of
vector size 5

[- - - -]

* The bigger window size the better model
will perform



* Reverse of CBOW

↳ Skip-gram: Word 2 Vec:

* All steps are same as "CBOW" just swap input & output columns.

→ iNeuron Company is Related to Data Science.

Window size = 5

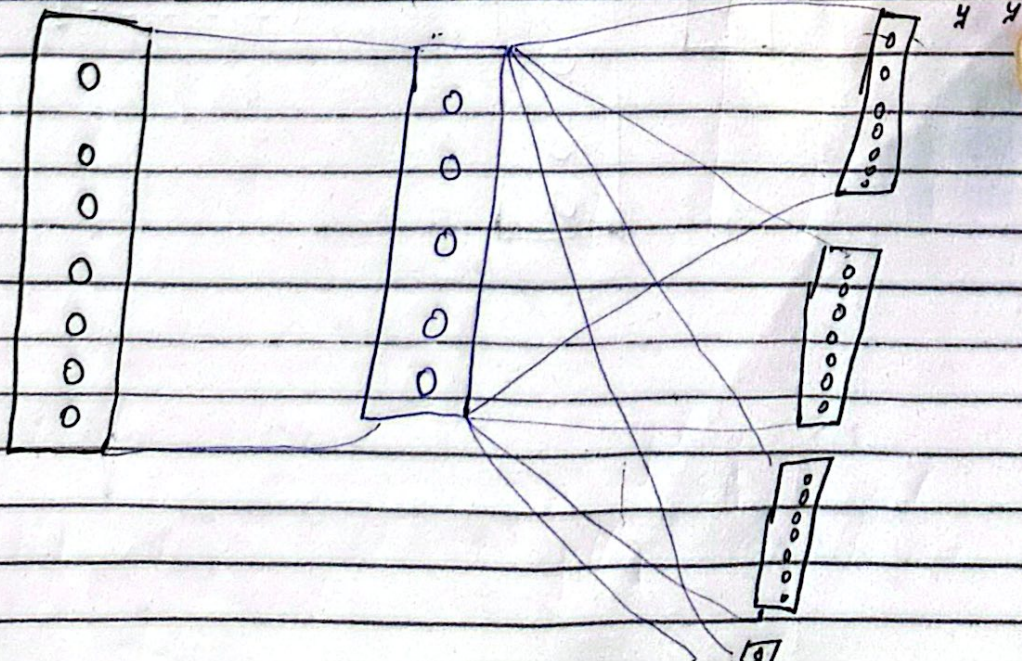
* Here middle word will become input & rest output

O/p	I/N
ineuron, company, related, to	is
company, is, to, data	Related
is, related, Data, science	to

* Input Layer
(7-Dimension Vector)

* Hidden Layer
(window size = 5)

* Output Layer
(7-Vector size)



↳ When to apply CBOW & Skipgram:

- * Small Dataset $\rightarrow$ CBOW
- * Large Dataset $\rightarrow$ skip-gram

↳ How to improve CBOW & Skipgram:

- * Increase the Training Data
- * Increase the Window size (which in turn increase Dimensions).

↳ Advantages of Word2Vec:

- * No sparse Matrix $\Rightarrow$ Dense Matrix
- * Semantic Info is getting captured.
- * Vocabulary size $\rightarrow$ fixed set of Dimensions
- * Out of Vocabulary is also solved.

⇒ Arg Word 2 Vec :

* To solve classification problem we use this.

Text

o/p

1: The Food is good	1
2: The Food is Bad	0
3: The Food is Amazing	1

* Now just like before we will use word2Vec to convert the sentences into vector (i.e. 300 dimension vector).

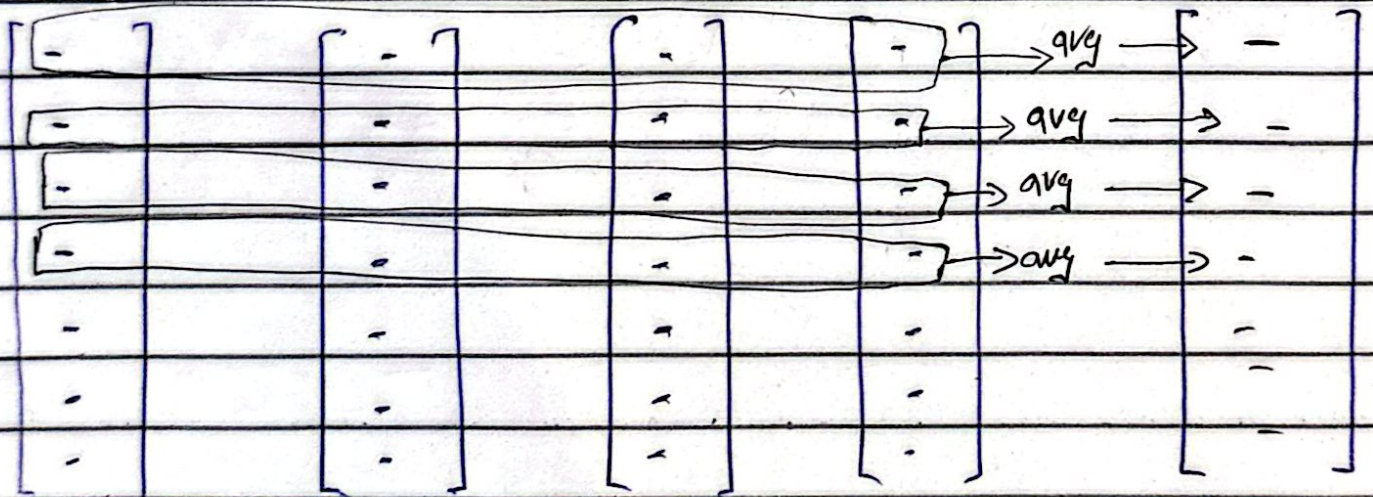
1: [The Food is good]

$\begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$ $\begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$ $\begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$ $\begin{bmatrix} - \\ - \\ - \\ - \\ - \end{bmatrix}$

↓
nth dimension Vector
(300)

* In the sentence 1 I am getting "300 dimension Vector" for each word but I should be getting "300 dimension vector" for whole sentence.

The Food is good Word sentence
vector



Take avg of one row for every vector
& write it.